

PROJECT BASED LEARNING

DATABASE MANAGEMENT SYSTEM



Submitted by:

Rutwik Reddy -25WU0102231

Specialization: AIML Section: Panthers
(B. Tech2025-29)

School of Technology

Submitted on: 6/10/2026

MULTI-WAREHOUSE STOCK TRANSFER MANAGEMENT SYSTEM

A Database Management System Project Report

COVER PAGE

MULTI-WAREHOUSE STOCK TRANSFER MANAGEMENT SYSTEM

A Project Report submitted in partial fulfilment of the requirements for the course
Database Management Systems

Submitted by	[STUDENT NAME]
Register / Roll Number	[ROLL NUMBER]
Course	B.Tech – Database Management Systems
Department	[DEPARTMENT]
University	[UNIVERSITY NAME]
Academic Year	[ACADEMIC YEAR]
Project Guide	[FACULTY NAME]

CERTIFICATE

This is to certify that the project report entitled "**Multi-Warehouse Stock Transfer Management System**" is a bonafide record of work carried out by [STUDENT NAME] (Roll No. [ROLL NUMBER]) under the guidance of [FACULTY NAME], in the Department of [DEPARTMENT], [UNIVERSITY NAME], during the academic year [ACADEMIC YEAR], in partial fulfilment of the requirements of the Database Management Systems course. The work embodied in this report has not been submitted to any other university or institution for the award of any degree or diploma.

Project Guide: [FACULTY NAME]	Head of Department: [HOD NAME]
Signature: _____	Signature: _____

Date: _____	External Examiner: _____
-------------	--------------------------

DECLARATION

I, [STUDENT NAME], Roll No. [ROLL NUMBER], hereby declare that the project report entitled "**Multi-Warehouse Stock Transfer Management System**" is an original work carried out by me under the guidance of [FACULTY NAME], Department of [DEPARTMENT], [UNIVERSITY NAME].

This work has not been copied from any other source, and it has not been submitted earlier, in part or in full, to any other university or institution for any degree, diploma or course. All sources of information used in this report have been duly acknowledged.

Place: _____

Date: _____

Signature: _____ ([STUDENT NAME])

ACKNOWLEDGEMENT

I express my sincere gratitude to [UNIVERSITY NAME] for giving me the opportunity and the facilities to carry out this project.

I am deeply thankful to my project guide, [FACULTY NAME], for the valuable guidance, constant encouragement and constructive suggestions during every stage of this work. I also thank the Head and all the faculty members of the Department of [DEPARTMENT] for their support and for the knowledge they have shared with me throughout the course. I thank my friends and classmates for their help, discussions and honest feedback while testing the system. Finally, I am grateful to my family for their patience, motivation and support, without which this project would not have been possible.

[STUDENT NAME]

ABSTRACT

Managing inventory in a business that operates several warehouses is far more difficult than managing a single store room. Stock quantities change every day because of purchases, sales and, most importantly, movement of goods from one warehouse to another. When these transfers are recorded on paper or in separate spreadsheets, the

records quickly become inaccurate. Items are counted twice, transfers are delayed, nobody knows which goods are currently in transit, and managers cannot see the real stock position of the organisation.

This project proposes the **Multi-Warehouse Stock Transfer Management System**, a browser-based database application that manages stock across multiple warehouses and controls every stock transfer through a defined life cycle. The system uses **MySQL** as the central database, **Python (Flask)** as the backend and **HTML, CSS and JavaScript** for the user interface (technology choices for the frontend framework are stated as assumptions in this report).

A transfer begins when a user creates a request from a source warehouse to a destination warehouse. The system validates that sufficient stock exists, and an authorised manager approves or rejects the request. After approval the goods are dispatched, and the stock is deducted from the source warehouse and shown as *in transit*. When the destination warehouse confirms receipt, the stock is added to the destination warehouse. All of these steps are recorded in transaction and audit tables, and database transactions guarantee that stock is never deducted without being added, or the reverse.

The database is designed up to the Third Normal Form, with primary keys, foreign keys and constraints that protect data integrity. Role-based access separates the powers of administrators, warehouse managers and inventory staff.

The expected outcome is a single, accurate source of truth for stock, faster and more transparent transfers, fewer human errors, a complete movement history and better reports for decision-making.

TABLE OF CONTENTS

Chapter	Title
–	Cover Page, Certificate, Declaration, Acknowledgement, Abstract
–	List of Figures, List of Tables
1	Introduction
2	Literature Review / Existing System
3	Requirement Analysis
4	System Analysis and Design (Architecture, Workflow, Use Case, DFD)

Chapter	Title
5	ER Diagram and Database Design (Normalization)
6	Implementation (Frontend, Backend, MySQL)
7	SQL Implementation
8	User Interface
9	Testing
10	Results and Discussion
11	Security
12	Advantages and Limitations
13	Future Enhancements
14	Conclusion
–	References
–	Appendix
A-I	Project Summary, Viva Q&A, Demo Flow, Presentation Scripts, Faculty Notes, Defence of Limitations, SQL Viva Questions

LIST OF FIGURES

Figure	Title
Fig 4.1	System Architecture
Fig 4.2	Stock Transfer Workflow
Fig 4.3	Use Case Diagram
Fig 4.4	Context-Level DFD
Fig 4.5	Level-1 DFD
Fig 4.6	ER Diagram
Fig 4.7	Database Schema / Relationship Diagram
Fig 4.8	Website Dashboard [INSERT SCREENSHOT HERE]

Fig 5.1

Fig 5.2

Fig 8.1

Figur	Title
e Fig 8.2	Warehouse Management Page [INSERT SCREENSHOT HERE]
Fig 8.3	Stock Management Page [INSERT SCREENSHOT HERE]
Fig 8.4	Transfer Management Page [INSERT SCREENSHOT HERE]
Fig 8.5	Reports / Dashboard [INSERT SCREENSHOT HERE]

LIST OF TABLES

Tabl	Title
e 2.1	Manual vs Existing Digital vs Proposed
3.1	System Hardware Requirements
3.2	Software Requirements
5.1	users
5.2	warehouses
5.3	categories
5.4	suppliers
5.5	products
5.6	stock
5.7	stock_transfers
5.8	transfer_items
5.9	stock_transactions
5.10	audit_logs
9.1	Test Cases

Note on assumptions. Details not confirmed by the project owner are marked **(Assumption)**. Features are described as *implemented* only when they form the core design of this system; everything else is listed under Future Enhancements (Chapter 13).

CHAPTER 1 – INTRODUCTION

1.1 Background

Companies that sell or distribute goods usually keep their stock in more than one warehouse, for example one near the city, one near the port and one in a regional hub. Goods are often moved between these warehouses to meet demand, to balance stock levels or to prepare for seasonal sales. Such movement is called a **stock transfer**.

A Database Management System (DBMS) stores data in an organised, shared and secure way. Using a DBMS such as MySQL, all warehouse data can be kept in one place, and every user sees the same up-to-date information.

1.2 Problem Statement

When stock transfers between warehouses are recorded manually, there is no reliable way to know how much stock each warehouse actually holds, which transfers are pending or in transit, and who approved what. There is a need for a centralised system that controls the full transfer process and keeps stock quantities accurate automatically.

1.3 Motivation

- Stock transfers are a daily activity in multi-warehouse businesses, but are often handled through phone calls, paper slips and spreadsheets.
- A single wrong entry can lead to stock-outs in one warehouse and excess stock in another.
- The project gives practical experience of applying DBMS concepts: ER modelling, normalization, constraints, joins, and transactions.

1.4 Objectives

1. To maintain a centralised database of warehouses, products and stock.
2. To allow users to create stock transfer requests between warehouses.
3. To validate stock availability before a transfer is accepted.
4. To provide an approval/rejection step for authorised managers.

5. To track each transfer through the states *Pending, Approved, Dispatched (In Transit), Received, Rejected and Cancelled*.
6. To update source and destination stock automatically and safely using database transactions.
7. To maintain a complete transaction history and audit log.
8. To provide dashboards and reports for decision-making.

1.5 Scope of the Project

In scope: user login with roles, warehouse, category, product and supplier management, stock management per warehouse, stock transfer life cycle, transfer history, search and filtering, dashboard, basic reports and audit logs.

Out of scope (basic version): purchase orders, sales billing, barcode/RFID scanning, demand forecasting, mobile application and ERP integration (see Chapter 13).

1.6 Importance of the System

The system gives management a real-time, accurate view of stock in all locations, reduces the time needed to move goods between warehouses and creates accountability because every action is recorded against a user and a time.

1.7 Target Users

User	Use of the System
Administrator	Manages users, warehouses, products; views all reports and logs
Warehouse Manager	Approves or rejects transfers for their warehouse; dispatches and receives stock
Inventory Staff	Updates stock, creates transfer requests, receives goods
Authorized User	Views stock and transfer status (read-only or limited rights)

1.8 Limitations

- The basic version does not use barcode or RFID scanning, so quantities depend on correct manual entry.
- Forecasting and automatic reorder suggestions are not included.

- A network connection to the server is required to use the web application.
- Transport cost and vehicle management are not covered.

1.9 Expected Outcomes

- A working web application connected to a MySQL database.
- Accurate stock per warehouse at all times.
- Every transfer traceable from request to receipt.
- Reduced paperwork and fewer manual errors.

Why Manual Management of Multiple Warehouses Causes Problems

Problem	Explanation	How the proposed system helps
Stock inaccuracies	Quantity is changed in one register but forgotten in another	Stock is updated automatically by the system
Duplicate records	Same product entered with different names in different sheets	Unique product code and foreign keys
Delayed transfers	Approval depends on phone calls and papers	Online approval queue
Poor visibility	Nobody knows what is in transit	Status tracking and In-Transit quantity
Human errors	Typing errors and wrong calculations	Validation and constraints
Difficult tracking of movement	No history of who moved what	Transaction history and audit log
No centralised information	Data is scattered across files	One central MySQL database

CHAPTER 2 – LITERATURE REVIEW / EXISTING SYSTEM

2.1 Existing Inventory Systems

Inventory systems range from simple notebooks and spreadsheets to large Enterprise Resource Planning (ERP) and Warehouse Management Systems (WMS). Small and medium businesses commonly use spreadsheets, while large organisations use commercial ERP software. (General description; no specific product is evaluated in this report.)

2.2 Traditional / Manual Warehouse Management

In the manual method, a storekeeper writes stock entries in a ledger. For a transfer, a paper slip is prepared, signed and physically carried or sent by message. The receiving warehouse writes the quantity in its own ledger when the goods arrive. The two ledgers are compared only occasionally.

2.3 Problems with Existing Approaches

- Records are updated late or not at all.
- Paper slips may be lost or damaged.
- No automatic check of available stock.
- Reports must be prepared by hand and are often out of date.
- Spreadsheets can be edited by anyone, with no audit trail.
- Different warehouses may use different formats.

2.4 Existing Digital Solutions

Spreadsheets and standalone single-location inventory software improve record keeping but usually treat each warehouse as a separate file or database. Large ERP systems support multiple locations but are expensive and complex for small organisations and for learning purposes.

2.5 Gaps in Existing Systems

1. No controlled approval workflow for inter-warehouse transfers in simple tools.
2. Weak tracking of goods in transit.
3. Lack of automatic, transaction-safe stock updates at both ends.
4. Poor audit history.
5. High cost and complexity of enterprise systems.

2.6 Proposed Improvement

The proposed system focuses specifically on the **transfer process**. It combines a central relational database, a defined status workflow, role-based approval, automatic stock updates inside database transactions and a full history of movements.

Table 2.1 – Comparison of Systems

Feature	Manual System	Existing Digital System (spreadsheet / single-location software)	Proposed Multi-Warehouse System
Data storage	Paper ledgers	Separate files	Central MySQL database
Multi-warehouse view	Not available	Limited	Available
Stock availability check	Manual	Manual / partial	Automatic validation
Transfer approval	Paper signature	Informal	Role-based online approval
In-transit tracking	None	Rarely	Status and in-transit quantity
Stock update	Manual at both ends	Manual	Automatic, transaction-safe
Audit trail	None	Weak	Transaction and audit logs
Reports	Hand-made	Manual formulas	Generated from queries
Error chance	High	Medium	Low
Cost	Low	Low–Medium	Low (open-source stack)

CHAPTER 3 – REQUIREMENT ANALYSIS

3.1 Functional Requirements

ID	Requirement	Description
FR1	User login	Users log in with username and password
FR2	User management	Admin creates, edits and deactivates users and assigns roles
FR3	Warehouse management	Add, edit, view warehouses with location and manager
FR4	Product management	Add, edit, view products with unit and reorder level
FR5	Category management	Maintain product categories
FR6	Stock management	Record stock quantity of each product in each warehouse
FR7	Stock availability check	System checks available quantity before a transfer is created
FR8	Create stock transfer	Choose source, destination, products and quantities
FR9	Approve / reject transfer	Authorised manager approves or rejects with remarks
FR10	Dispatch transfer	Source warehouse dispatches; stock is deducted
FR11	Receive transfer	Destination warehouse confirms receipt; stock is added
FR12	Automatic stock updates	Stock changes happen in a database transaction
FR13	Transfer history	View all past transfers with status and dates
FR14	Search / filter	Filter by warehouse, product, status and date
FR15	Dashboard	Summary of stock, pending and in-transit transfers
FR16	Reports	Stock by warehouse, low stock, transfer reports
FR17	Audit logs	Record who did which action and when

3.2 Non-Functional Requirements

Requirement	Description
Security	Hashed passwords, role-based access, parameterised queries
Performance	Common pages should load within a few seconds for normal data volumes; indexes on key columns
Reliability	Transactions ensure no partial stock updates
Scalability	New warehouses and products can be added without changing the design
Usability	Simple forms, clear status labels, readable tables
Maintainability	Modular Flask routes and a documented schema
Data integrity	Primary keys, foreign keys, CHECK and UNIQUE constraints
Availability	Accessible through a browser during working hours; regular database backups

3.3 Hardware Requirements (Assumption)

Component	Minimum
Processor	Intel Core i3 or equivalent
RAM	4 GB (8 GB recommended)
Storage	10 GB free space
Display	1366 × 768 or better
Network	LAN / Internet connection

3.4 Software Requirements

Software	Purpose
Operating system	Windows 10/11, Linux or macOS (Assumption)
Python 3 with Flask	Backend

Software	Purpose
MySQL Server	Database
HTML, CSS, JavaScript (React optional)	Frontend
VS Code	Code editor
MySQL Workbench / MySQL Command Line	Database administration
Git and GitHub	Version control
Web browser (Chrome, Edge, Firefox)	Running the application

3.5 User Requirements

- **Admin:** full control of users, masters and reports.
- **Warehouse Manager:** a clear list of transfers waiting for approval, dispatch or receipt.
- **Inventory Staff:** quick forms to update stock and request transfers.
- **Authorized User:** read-only view of stock and transfer status.

CHAPTER 4 – SYSTEM ANALYSIS AND DESIGN

4.1 System Architecture

The system follows a three-tier architecture:

User → Frontend (browser) → Backend (Flask) → Database (MySQL) → Response

1. The user opens the web page and enters data.
2. The frontend sends an HTTP request (JSON) to the Flask backend.
3. The backend checks the session, role and input, then runs SQL queries.
4. MySQL stores or returns the data.
5. The backend sends a response, and the frontend shows the result.

Fig 4.1 – System Architecture

flowchart LR

```

U[User] --> F[Frontend: HTML / CSS / JavaScript]
F -->|HTTP request JSON| B[Backend: Python Flask]
B -->|SQL queries / transactions| D[(MySQL Database)]
D -->|Result set| B
B -->|JSON response| F
F --> U

```

4.2 System Workflow

Source Warehouse → Transfer Request → Validation → Approval → Dispatch → In-Transit → Destination Warehouse → Receive → Stock Update → Transaction History

Step	Status	Action	Stock effect
1	Pending	Staff creates request	None
2	Pending	System validates available stock	None
3	Approved / Rejected	Manager decides	None
4	Dispatched (In Transit)	Source dispatches goods	Source stock decreases
5	Received	Destination confirms receipt	Destination stock increases
6	Received	History and audit entries saved	Transaction rows created

A transfer may also be **Cancelled** by the requester while it is still Pending.

Fig 4.2 – Stock Transfer Workflow

flowchart TD

```

A[Source Warehouse] --> B[Create Transfer Request]
B --> C{Validate stock available?}
C -- No --> X[Show Insufficient Stock error]
C -- Yes --> D[Status: Pending]
D --> E{Manager decision}
E -- Reject --> R[Status: Rejected]
E -- Approve --> F[Status: Approved]
F --> G[Dispatch]
G --> H[Deduct stock from source - Status: In Transit]
H --> I[Destination Warehouse]
I --> J[Receive]
J --> K[Add stock to destination - Status: Received]
K --> L[Write Transaction History and Audit Log]

```

4.3 Use Case Diagram

Actors: Admin, Warehouse Manager, Inventory Staff, Authorized User.

Use Case	Admin	Warehouse Manager	Inventory Staff	Authorized User
Login / Logout	✓	✓	✓	✓
Manage users	✓			
Manage warehouses	✓			
Manage products and categories	✓	✓		
Update stock	✓	✓	✓	
Create transfer request	✓	✓	✓	
Approve / reject transfer	✓	✓		
Dispatch transfer	✓	✓	✓	
Receive transfer	✓	✓	✓	
View transfer history	✓	✓	✓	✓
View reports / dashboard	✓	✓	✓	✓
View audit logs	✓			

Fig 4.3 – Use Case Diagram

flowchart LR

Admin([Admin])

WM([Warehouse Manager])

IS([Inventory Staff])

AU([Authorized User])

subgraph System[Multi-Warehouse Stock Transfer Management System]

UC1(Login / Logout)

UC2(Manage Users)

UC3(Manage Warehouses)

UC4(Manage Products and Categories)

UC5(Update Stock)

UC6(Create Transfer Request)

UC7(Approve / Reject Transfer)

UC8(Dispatch Transfer)

UC9(Receive Transfer)

UC10(View Transfer History)

UC11(View Reports and Dashboard)

UC12(View Audit Logs)

end

Admin --> UC1 & UC2 & UC3 & UC4 & UC12
 WM --> UC1 & UC4 & UC5 & UC7 & UC8 & UC9
 IS --> UC1 & UC5 & UC6 & UC8 & UC9
 AU --> UC1 & UC10 & UC11
 Admin --> UC7 & UC11

4.4 Data Flow Diagrams

Fig 4.4 – Context-Level DFD (Level 0)

flowchart LR

A[Admin] -->|Master data, user details| S((Multi-Warehouse Stock Transfer System))
 M[Warehouse Manager] -->|Approval, dispatch, receive| S
 I[Inventory Staff] -->|Stock updates, transfer requests| S
 S -->|Reports, status, history| A
 S -->|Pending list, notifications on screen| M
 S -->|Stock and transfer details| I
 U[Authorized User] -->|View requests| S
 S -->|Read-only information| U

Fig 4.5 – Level-1 DFD

flowchart TD

U[Users] -->|credentials| P1((1.0 Authentication))
 P1 <--> D1[(users)]
 U -->|master data| P2((2.0 Manage Warehouses, Products, Categories))
 P2 <--> D2[(warehouses / products / categories)]
 U -->|stock entries| P3((3.0 Manage Stock))
 P3 <--> D3[(stock)]
 U -->|transfer request| P4((4.0 Create and Validate Transfer))
 P4 --> D3
 P4 --> D4[(stock_transfers / transfer_items)]
 U -->|decision| P5((5.0 Approve / Reject))
 P5 <--> D4
 U -->|dispatch / receive| P6((6.0 Dispatch and Receive))
 P6 <--> D4
 P6 --> D3
 P6 --> D5[(stock_transactions)]
 P1 --> D6[(audit_logs)]
 P5 --> D6
 P6 --> D6

```

D3 --> P7((7.0 Reports and Dashboard))
D4 --> P7
D5 --> P7
P7 -->|reports| U

```

CHAPTER 5 – ER DIAGRAM AND DATABASE DESIGN

5.1 ER Diagram

The Entity-Relationship (ER) model shows the main entities of the system and how they are related.

Entities: User, Warehouse, Category, Supplier, Product, Stock, Stock Transfer, Transfer Item, Stock Transaction, Audit Log.

Fig 5.1 – ER Diagram

```

erDiagram
    USERS ||--o{ WAREHOUSES : manages
    CATEGORIES ||--o{ PRODUCTS : classifies
    SUPPLIERS ||--o{ PRODUCTS : supplies
    WAREHOUSES ||--o{ STOCK : holds
    PRODUCTS ||--o{ STOCK : "stored as"
    WAREHOUSES ||--o{ STOCK_TRANSFERS : "is source of"
    WAREHOUSES ||--o{ STOCK_TRANSFERS : "is destination of"
    USERS ||--o{ STOCK_TRANSFERS : "requests / approves / dispatches/
receives"
    STOCK_TRANSFERS ||--|{ TRANSFER_ITEMS : contains
    PRODUCTS ||--o{ TRANSFER_ITEMS : "appears in"
    STOCK_TRANSFERS ||--o{ STOCK_TRANSACTIONS : generates
    PRODUCTS ||--o{ STOCK_TRANSACTIONS : "moved in"
    WAREHOUSES ||--o{ STOCK_TRANSACTIONS : "affected"
    USERS ||--o{ STOCK_TRANSACTIONS : performs
    USERS ||--o{ AUDIT_LOGS : "creates"

    USERS {
        int user_id PK
        varchar username UK
        varchar password_hash
        varchar full_name
        enum role
    }
    WAREHOUSES {
        int warehouse_id PK
        varchar warehouse_code UK

```

```

        varchar warehouse_name
        intmanager_id FK
    }
    CATEGORIES {
        intcategory_id PK
        varcharcategory_name UK
    }
    SUPPLIERS {
        intsupplier_id PK
        varchar supplier_name
    }
    PRODUCTS {
        intproduct_id    PK
        varcharsku        UK
        intcategory_id   FK
        intsupplier_id   FK
        int reorder_level
    }
    STOCK {
        intstock_id PK
        intwarehouse_id FK
        intproduct_id FK
        int quantity
    }
    STOCK_TRANSFERS {
        inttransfer_id PK
        intsource_warehouse_id FK
        intdestination_warehouse_id FK
        enum status
    }
    TRANSFER_ITEMS {
        inttransfer_item_id PK
        inttransfer_id FK
        intproduct_id FK
        int quantity
    }
    STOCK_TRANSACTIONS {
        inttransaction_id PK
        intproduct_id FK
        intwarehouse_id FK
        inttransfer_id FK
        int quantity_change
    }
    AUDIT_LOGS {

```

```

    int log_id PK
    int user_id FK
    varchar action
}

```

5.2 Keys and Relationships

Primary keys: each table has a single auto-increment integer primary key (for example `warehouse_id`, `product_id`).

Foreign keys: listed in the table below.

Child table	Foreign	Parent table	Meaning
warehouses	key manager_id	users	Manager of the warehouse
products	category_id	categories	Category of the product
products	supplier_id	suppliers	Main supplier of the product
stock	warehouse_id	warehouses	Where the stock is kept
stock	product_id	products	Which product it is
stock_transfers	source_warehouse_id, destination_warehouse_id	warehouses	From and to warehouses
stock_transfers	requested_by, approved_by, dispatched_by, received_by	users	Users involved in each step
transfer_items	transfer_id	stock_transfers	Parent transfer
transfer_items	product_id	products	Product being moved
stock_transactions	product_id, warehouse_id, transfer_id, created_by	products, warehouses, stock_transfers, users	Movement history

Child table	Foreign key	Parent table	Meaning
audit_logs	user_id	users	User who performed the action

One-to-many relationships: User–Warehouse (one manager manages one or more warehouses), Category–Product, Supplier–Product, Warehouse–Stock, Product–Stock, Stock Transfer–Transfer Item, Warehouse–Stock Transfer (twice: as source and as destination), User–Audit Log.

One-to-one relationships: none are required in this design. A user's role is stored as a column of users, so no separate one-to-one table is needed.

Many-to-many relationships:

1. **Warehouse ↔ Product** is many-to-many (a warehouse stores many products, and a product is stored in many warehouses). It is resolved by the associative table `stock`, which also holds the attribute `quantity`.
2. **Stock Transfer ↔ Product** is many-to-many (a transfer can carry many products, and a product can appear in many transfers). It is resolved by the associative table `transfer_items`, which holds `quantity`.

Fig 5.2 – Database Relationship Diagram

```

flowchart LR
    users --> warehouses
    categories --> products
    suppliers --> products
    warehouses --> stock
    products --> stock
    warehouses --> stock_transfers
    users --> stock_transfers
    stock_transfers --> transfer_items
    products --> transfer_items
    stock_transfers --> stock_transactions
    products --> stock_transactions
    warehouses --> stock_transactions
    users --> audit_logs
  
```

(Arrow direction: parent table → child table that holds the foreign key.)

5.3 Database Design – Table Structures

The database is named `warehouse_transfer_db` (Assumption: name chosen for this report). Data types follow MySQL 8.0.

Table 5.1 –users

Attribute	Data type	Constraints	Description
user_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user ID
username	VARCHAR(50)	NOT NULL,	Login name
password_hash	VARCHAR(255)	UNIQUE NOT NULL	Hashed password
full_name	VARCHAR(100)	NOT NULL	Name of the user
email	VARCHAR(100)	UNIQUE	Email address
role	ENUM('ADMIN','MANAGER','STAFF','VIEWER')	NOT NULL, DEFAULT 'VIEWER'	Role for access control
is_active	TINYINT(1)	NOT NULL, DEFAULT 1	Active or disabled account
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Creation time

Table 5.2 –warehouses

Attribute	Data type	Constraints	Description
warehouse_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique warehouse ID
warehouse_code	VARCHAR(10)	NOT NULL,	Short code, e.g. WH-DEL
warehouse_name	VARCHAR(100)	UNIQUE NOT NULL	Warehouse name
city	VARCHAR(50)	NOT NULL	City
address	VARCHAR(255)	NOT NULL	Full address
manager_id	INT	FOREIGN KEY → users(user_id)	Warehouse manager
is_active	TINYINT(1)	NOT NULL, DEFAULT 1	Active status

Table 5.3 – categories

Attribute	Data type	Constraints	Description
category_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique category ID
category_name	VARCHAR(50)	NOT NULL,	e.g. Electronics
description	VARCHAR(255)	UNIQUE	Optional notes

Table 5.4 – suppliers

Attribute	Data type	Constraints	Description
supplier_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique supplier ID
supplier_name	VARCHAR(100)	NOT NULL	Supplier name
phone	VARCHAR(20)		Contact number
email	VARCHAR(100)		Contact email

Table 5.5 – products

Attribute	Data type	Constraints	Description
product_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique product ID
sku	VARCHAR(30)	NOT NULL, UNIQUE	Stock keeping unit code
product_name	VARCHAR(100)	NOT NULL	Product name
category_id	INT	NOT NULL, FOREIGN KEY → categories	Category
supplier_id	INT	FOREIGN KEY → suppliers	Supplier
unit	VARCHAR(20)	NOT NULL, DEFAULT 'PCS'	Unit of measure
reorder_level	INT	NOT NULL, DEFAULT 10, CHECK (reorder_level >= 0)	Low-stock threshold

Table 5.6 –stock

Attribute	Data	Constraints	Description
stock_id	type INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID
warehouse_id	INT	NOT NULL, FOREIGN KEY → warehouses	Warehouse
product_id	INT	NOT NULL, FOREIGN KEY → products	Product
quantity	INT	NOT NULL, DEFAULT 0, CHECK (quantity >= 0)	Current quantity on hand
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last change time
		UNIQUE (warehouse_id, product_id)	One row per product per warehouse

Table 5.7 –stock_transfers

Attribute	Data type	Constraints	Description
transfer_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique transfer ID
transfer_code	VARCHAR(20)	NOT NULL, UNIQUE	Readable code, e.g. TR-2025-0001
source_warehouse_id	INT	NOT NULL, FOREIGN KEY → warehouses	From warehouse
destination_warehouse_id	INT	NOT NULL, FOREIGN KEY → warehouses	To warehouse
status	ENUM('PENDING','APPROVED','REJECTED','IN_TRANSIT','RECEIVED','CANCELLED')	NOT NULL, DEFAULT 'PENDING'	Current state
requested_by	INT	NOT NULL, FOREIGN KEY → users	Requester
approved_by	INT	FOREIGN KEY → users	Approver or rejecter
dispatched_by	INT	FOREIGN KEY → users	User who dispatched
received_by	INT	FOREIGN KEY → users	User who received
request_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Request time

Attribute	Data type	Constraints	Description
approval_date	DATETIME		Approval time
dispatch_date	DATETIME		Dispatch time
received_date	DATETIME		Receipt time
remarks	VARCHAR(255)		Notes / reason for rejection
		CHECK (source_warehouse_id <> destination_warehouse_id)	A warehouse cannot transfer to itself

Table 5.8 – transfer_items

Attribute	Data type	Constraints	Description
transfer_item_id		PRIMARY KEY, AUTO_INCREMENT	Unique ID
transfer_id	INT	NOT NULL, FOREIGN KEY → stock_transfers (ON DELETE CASCADE)	Parent transfer
product_id	INT	NOT NULL, FOREIGN KEY → products	Product moved
quantity	INT	NOT NULL, CHECK (quantity > 0)	Quantity to transfer
		UNIQUE (transfer_id, product_id)	A product appears once per transfer

Table 5.9 – stock_transactions

Attribute	Data type	Constraints	Description
transaction_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID
product_id	INT	NOT NULL, FOREIGN KEY → products	Product
warehouse_id	INT	NOT NULL, FOREIGN KEY → warehouses	Warehouse affected
transfer_id	INT	FOREIGN KEY → stock_transfers	Related transfer (NULL for manual stock entry)
txn_type	ENUM('STOCK_IN','TRANSFER_OUT','TRANSFER_IN','ADJUSTMENT')	NOT NULL	Type of movement
quantity_change	INT	NOT NULL	Positive for increase, negative for decrease

Attribute	Data type	Constraints	Description
created_by	INT	NOT NULL, FOREIGN KEY → users	User responsible
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time of movement

Table 5.10 –audit_logs

Attribute	Data type	Constraints	Description
log_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID
user_id	INT	FOREIGN KEY → users	User who acted (NULL for failed login of unknown user)
action	VARCHAR(50)	NOT NULL	e.g. LOGIN, CREATE_TRANSFER, APPROVE_TRANSFER
entity_type	VARCHAR(30)		e.g. TRANSFER, PRODUCT
entity_id	INT		ID of the affected record
details	VARCHAR(255)		Short description
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time of action

Design note. Stock is deducted from the source warehouse when the transfer is **dispatched**, and added to the destination warehouse when it is **received**. The quantity currently in transit is therefore not part of any warehouse's quantity; it is calculated from transfer_items of transfers whose status is IN_TRANSIT.

5.4 Normalization

Normalization removes redundancy and anomalies by organising data into well-structured tables.

Unnormalized example. Imagine one sheet with the columns: *TransferNo*, *SourceWarehouseName*, *SourceCity*, *DestinationWarehouseName*, *ProductNames* (many in one cell), *Quantities* (many in one cell), *ProductCategory*, *SupplierName*, *RequestedByName*.

First Normal Form (1NF). Every column holds a single atomic value and there are no repeating groups. The multiple products in one cell are moved into separate rows, which become the `transfer_items` (one row per product per transfer).

Second Normal Form (2NF). The table is in 1NF and every non-key attribute depends on the whole primary key. In `transfer_items`, the key is `(transfer_id, product_id)`. Product name, category and supplier depend only on `product_id`, so they are moved to `products`. Warehouse name and city depend only on the warehouse, so they are kept in `warehouses`. Stock quantity depends on both warehouse and product, so it is kept in `stock` with `(warehouse_id, product_id)` as a unique pair.

Third Normal Form (3NF). The table is in 2NF and has no transitive dependency (non-key attribute depending on another non-key attribute). For example, in `products`, `category_name` would depend on `category_id`, not on `product_id`, so categories are stored in `categories` and referenced by a foreign key. Supplier details are likewise stored in `suppliers`.

All tables of this design satisfy 3NF.

5.5 How the Design Avoids Anomalies

Anomaly	Meaning	How it is avoided
Data redundancy	Same data stored many times	Product, warehouse, category and supplier details are stored once and referenced by IDs
Update anomaly	Changing a value in one row but not others	A product name or warehouse city is changed in one row only
Insert anomaly	Cannot add data without unrelated data	A new warehouse or product can be added before any stock or transfer exists
Delete anomaly	Deleting a row loses unrelated facts	Deleting a transfer does not delete product or warehouse details; foreign keys restrict deletion of parents still in use

Note on `stock_transactions`. Historical transaction rows store quantities as they happened. This is deliberate history keeping and not redundancy, because the values cannot be derived from the current state of other tables.

CHAPTER 6 – IMPLEMENTATION

Before submission: this chapter describes the system as designed in this report. Check each statement against your actual code and remove or move to Chapter 13 anything you have not built.

6.1 Frontend

The frontend runs in the browser and is built with HTML, CSS and JavaScript (React may be used instead of plain JavaScript – **Assumption**). It sends requests to the Flask backend using the `fetch()` API and shows the returned data in tables and forms.

Page	Description
Login page	Username and password form; shows an error for wrong credentials
Dashboard	Cards for total products, total stock, pending transfers, in-transit transfers and low-stock items
Warehouse page	List of warehouses; add and edit forms (Admin)
Product page	List of products with category and supplier; add and edit forms
Inventory page	Stock of each product in the selected warehouse; search and filter; low-stock highlight
Stock transfer page	Form to choose source, destination, products and quantities; shows available stock
Transfer tracking page	List of transfers with status; buttons for Approve, Reject, Dispatch and Receive depending on role and status
Reports page	Stock by warehouse, low-stock report, transfer report with date filter

6.2 Backend

The backend is a Python Flask application. It receives requests, checks that the user is logged in and has the right role, validates the input, runs SQL through a MySQL connector library (**Assumption:** `mysql-connector-python`) and returns JSON.

Server. `app.py` creates the Flask app, loads the database settings from a configuration file or environment variables (so passwords are not written in the source code) and registers the routes.

Routes / APIs.

Metho	Endpoint	Purpose	Allowed roles
d POST	/api/login	Log in	All
POST	/api/logout	Log out	All
GET, POST	/api/warehouse s	List / add warehouses	Admin (add); all (list)
GET, POST	/api/products	List / add products	Admin, Manager (add)
GET	/api/stock?warehouse_id=	Stock of a warehouse	All
POST	/api/stock/add	Add stock to a warehouse	Admin, Manager,
POST	/api/transfers	Create a transfer request	Staff Admin, Manager,
GET	/api/transfers?status=	List / filter transfers	Staff
POS	/api/transfers/<id>/approve	Approve	All
T	/api/transfers/<id>/reject	Reject	Admin, Manager
POS T	/api/transfers/<id>/dispatch	Dispatch	Admin, Manager Admin, Manager,
POST POS	/api/transfers/<id>/receive	Receive	Staff
T GET	/api/reports/stock-by- warehouse	Report	Admin, Manager, Staff
GET	/api/reports/low-stock	Report	All
GET	/api/audit-logs	View logs	Admin

Authentication. On login, the backend finds the user by username and compares the entered password with the stored hash using a hashing library (**Assumption:** Werkzeug's `check_password_hash`). On success, the user ID and role are stored in the Flask session.

Validation. Before any SQL runs, the backend checks that required fields are present, quantities are positive whole numbers, the source and destination warehouses are different and exist, and every product exists.

Business logic. The main rules are:

1. A transfer can be created only if the source warehouse has enough stock for every item.
2. Only a transfer in `PENDING` status can be approved, rejected or cancelled.
3. Only an `APPROVED` transfer can be dispatched.
4. Only an `IN_TRANSIT` transfer can be received.
5. Dispatch and receive each run inside a single database transaction.

Error handling. The backend uses `try / except` blocks. If any step fails, the transaction is rolled back and a clear JSON error message with a suitable HTTP status code (400 for bad input, 401 for not logged in, 403 for not allowed, 404 for not found, 500 for server error) is returned. Errors are not shown to the user with technical details.

Sample code snippet – login.

```
@app.route("/api/login", methods=["POST"])
def login():
    data = request.get_json()
    username = data.get("username", "").strip()
    password = data.get("password", "")

    cur = db.cursor(dictionary=True)
    cur.execute(
        "SELECT user_id, password_hash, role FROM users "
        "WHERE username = %s AND is_active = 1",
        (username,)
    )
    user = cur.fetchone()

    if user and check_password_hash(user["password_hash"], password):
        session["user_id"] = user["user_id"]
        session["role"] = user["role"]
        write_audit(user["user_id"], "LOGIN", "USER", user["user_id"])
        return jsonify({"message": "Login successful", "role": user["role"]})

    return jsonify({"error": "Invalid username or password"}), 401
```

Sample code snippet – dispatch a transfer (with transaction).

```

@app.route("/api/transfers/<int:transfer_id>/dispatch", methods=["POST"])
@login_required
def dispatch_transfer(transfer_id):
    cur = db.cursor(dictionary=True)
    try:
        db.start_transaction()
        cur.execute(
            "SELECT source_warehouse_id FROM stock_transfers "
            "WHERE transfer_id = %s AND status = 'APPROVED' FOR UPDATE",
            (transfer_id,)
        )
        transfer = cur.fetchone()
        if not transfer:
            db.rollback()
            return jsonify({"error": "Transfer not found or not approved"}),
400

        cur.execute(
            "SELECT product_id, quantity FROM transfer_items WHERE
transfer_id = %s",
            (transfer_id,)
        )
        for item in cur.fetchall():
            cur.execute(
                "UPDATE stock SET quantity = quantity - %s "
                "%s",
                "WHERE warehouse_id = %s AND product_id = %s AND quantity >=
                (item["quantity"], transfer["source_warehouse_id"],
                item["product_id"], item["quantity"])
            )
            if cur.rowcount == 0:
                db.rollback()
                return jsonify({"error": "Insufficient stock at dispatch"}),
400

            cur.execute(
                "INSERT INTO stock_transactions "
                "(product_id, warehouse_id, transfer_id, txn_type,
quantity_change, created_by) "
                "VALUES (%s, %s, %s, 'TRANSFER_OUT', %s, %s)",
                (item["product_id"], transfer["source_warehouse_id"],
                transfer_id, -item["quantity"], session["user_id"])
            )

        cur.execute(
            "UPDATE stock_transfers SET status = 'IN_TRANSIT', "
            "dispatched_by = %s, dispatch_date = NOW() WHERE transfer_id =

```

```

%s",
        (session["user_id"], transfer_id)
    )
    db.commit()
    write_audit(session["user_id"], "DISPATCH_TRANSFER", "TRANSFER",
transfer_id)
    return jsonify({"message": "Transfer dispatched"})
except Exception:
    db.rollback()
    return jsonify({"error": "Could not dispatch transfer"}), 500

```

6.3 MySQL Database

- **Database creation:** the database `warehouse_transfer_db` is created with `CREATE DATABASE` and selected with `USE`.
- **Table creation:** ten tables are created in the order parent first, child later (users, warehouses, categories, suppliers, products, stock, stock_transfers, transfer_items, stock_transactions, audit_logs).
- **Relationships:** foreign keys connect child tables to parent tables (see Table 5.x in Chapter 5).
- **Constraints:** PRIMARY KEY, UNIQUE, NOT NULL, DEFAULT, CHECK and FOREIGN KEY constraints protect the data. For example, CHECK (quantity >= 0) on stock prevents negative stock, and UNIQUE (warehouse_id, product_id) prevents duplicate stock rows.
- **Queries:** SELECT queries with joins, grouping and subqueries drive the dashboard, inventory page and reports.
- **Transactions:** dispatch and receive are executed with `START TRANSACTION`, `COMMIT` and `ROLLBACK` so that stock and status always change together. The tables use the InnoDB engine, which supports transactions and foreign keys.

CHAPTER 7 – SQL IMPLEMENTATION

All commands below are written for MySQL 8.0 (CHECK constraints are enforced from MySQL 8.0.16).

7.1 CREATE DATABASE

```
CREATE DATABASE IF NOT EXISTS warehouse_transfer_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE warehouse_transfer_db;
```

7.2 CREATE TABLE (with constraints and foreign keys)

```
CREATE TABLE users (
  user_id          INT AUTO_INCREMENT PRIMARY KEY,
  username         VARCHAR(50) NOT NULL UNIQUE,
  password_hash    VARCHAR(255) NOT NULL,
  full_name        VARCHAR(100) NOT NULL,
  email            VARCHAR(100) UNIQUE,
  role             ENUM('ADMIN','MANAGER','STAFF','VIEWER') NOT NULL DEFAULT
  'VIEWER',
  is_active        TINYINT(1)  NOT NULL DEFAULT 1,
  created_at       TIMESTAMP  NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

```
CREATE TABLE warehouses (
  warehouse_id     INT AUTO_INCREMENT PRIMARY KEY,
  warehouse_code   VARCHAR(10) NOT NULL UNIQUE,
  warehouse_name   VARCHAR(100) NOT NULL,
  city             VARCHAR(50) NOT NULL,
  address          VARCHAR(255),
  manager_id       INT,
  is_active        TINYINT(1)  NOT NULL DEFAULT 1,
  CONSTRAINT fk_wh_manager FOREIGN KEY (manager_id) REFERENCES
  users(user_id)
) ENGINE=InnoDB;
```

```
CREATE TABLE categories (
  category_id      INT AUTO_INCREMENT PRIMARY KEY,
  category_name    VARCHAR(50) NOT NULL UNIQUE,
  description       VARCHAR(255)
) ENGINE=InnoDB;
```

```
CREATE TABLE suppliers (
  supplier_id      INT AUTO_INCREMENT PRIMARY KEY,
  supplier_name    VARCHAR(100) NOT NULL,
  phone            VARCHAR(20),
  email            VARCHAR(100)
) ENGINE=InnoDB;
```

```

CREATE TABLE products (
  product_id      INT AUTO_INCREMENT PRIMARY KEY,
  sku              VARCHAR(30) NOT NULL UNIQUE,
  product_name    VARCHAR(100) NOT NULL,
  category_id     INT NOT NULL,
  supplier_id     INT,
  unit            VARCHAR(20) NOT NULL DEFAULT 'PCS',
  reorder_level   INT NOT NULL DEFAULT 10,
  CONSTRAINT chk_reorder CHECK (reorder_level >= 0),
  CONSTRAINT fk_prod_cat FOREIGN KEY (category_id) REFERENCES
categories(category_id),
  CONSTRAINT fk_prod_sup FOREIGN KEY (supplier_id) REFERENCES
suppliers(supplier_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE stock (
  stock_id        INT AUTO_INCREMENT PRIMARY KEY,
  warehouse_id    INT NOT NULL,
  product_id      INT NOT NULL,
  quantity        INT NOT NULL DEFAULT 0,
  last_updated    TIMESTAMP NOT NULL DEFAULT
                  CURRENT_TIMESTAMP ON UPDATE
                  CURRENT_TIMESTAMP,
  CONSTRAINT chk_stock_qty CHECK (quantity >= 0),
  CONSTRAINT uq_stock UNIQUE (warehouse_id, product_id),
  CONSTRAINT fk_stock_warehouse FOREIGN KEY (warehouse_id) REFERENCES
warehouses(warehouse_id),
  CONSTRAINT fk_stock_product FOREIGN KEY (product_id) REFERENCES
products(product_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE stock_transfers (
  transfer_id     INT AUTO_INCREMENT PRIMARY KEY,
  source_warehouse_id VARCHAR(20) NOT NULL UNIQUE,
  destination_warehouse_id INT NOT NULL,
  status          ENUM('PENDING','APPROVED','REJECTED','IN_TRANSIT','RECEIVED','CANCELLED')
                  NOT NULL DEFAULT 'PENDING',
  requested_by    INT NOT NULL,
  approved_by     INT,
  dispatched_by   INT,
  received_by     INT,
  request_date    TIMESTAMP NOT NULL DEFAULT
                  CURRENT_TIMESTAMP,
  approval_date   DATETIME,
  dispatch_date   DATETIME,

```

```

received_date    DATETIME,
remarks          VARCHAR(255),
CONSTRAINT chk_diff_wh CHECK (source_warehouse_id <>
destination_warehouse_id),
CONSTRAINT fk_tr_src FOREIGN KEY (source_warehouse_id) REFERENCES
warehouses(warehouse_id),
CONSTRAINT fk_tr_dst FOREIGN KEY (destination_warehouse_id) REFERENCES
warehouses(warehouse_id),
CONSTRAINT fk_tr_req FOREIGN KEY (requested_by) REFERENCES
users(user_id),
CONSTRAINT fk_tr_app FOREIGN KEY (approved_by) REFERENCES
users(user_id),
CONSTRAINT fk_tr_disp FOREIGN KEY (dispatched_by) REFERENCES
users(user_id),
CONSTRAINT fk_tr_recv FOREIGN KEY (received_by) REFERENCES
users(user_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE transfer_items (
transfer_item_id INT AUTO_INCREMENT PRIMARY KEY,
transfer_id      INT NOT NULL,
product_id       INT NOT NULL,
quantity         INT NOT NULL,
CONSTRAINT chk_item_qty CHECK (quantity > 0),
CONSTRAINT uq_transfer_product UNIQUE (transfer_id, product_id),
CONSTRAINT fk_ti_transfer FOREIGN KEY (transfer_id)
REFERENCES stock_transfers(transfer_id) ON DELETE CASCADE,
CONSTRAINT fk_ti_product FOREIGN KEY (product_id) REFERENCES
products(product_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE stock_transactions (
transaction_id   INT AUTO_INCREMENT
product_id       PRIMARY KEY, INT NOT NULL,
warehouse_id     INT NOT NULL,
transfer_id      INT,
txn_type
ENUM('STOCK_IN','TRANSFER_OUT','TRANSFER_IN','ADJUSTMENT') NOT NULL,
quantity_change INT NOT NULL,
created_by       INT NOT NULL,
created_at       TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tx_prod FOREIGN KEY (product_id) REFERENCES
products(product_id),
CONSTRAINT fk_tx_wh FOREIGN KEY (warehouse_id) REFERENCES
warehouses(warehouse_id),
CONSTRAINT fk_tx_tr FOREIGN KEY (transfer_id) REFERENCES

```

```

stock_transfers(transfer_id),
    CONSTRAINT fk_tx_user FOREIGN KEY (created_by) REFERENCES
users(user_id)
) ENGINE=InnoDB;

CREATE TABLE audit_logs (
    log_id          INT AUTO_INCREMENT PRIMARY KEY,
    user_id         INT,
    action          VARCHAR(50) NOT NULL,
    entity_type     VARCHAR(30),
    entity_id       INT,
    details         VARCHAR(255),
    created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_audit_user FOREIGN KEY (user_id) REFERENCES users(user_id)
) ENGINE=InnoDB;

CREATE INDEX idx_transfer_status ON stock_transfers(status);
CREATE INDEX idx_tx_product ON stock_transactions(product_id, warehouse_id);

```

7.3 INSERT (sample data)

```

INSERT INTO users (username, password_hash, full_name, email, role) VALUES
('admin',      'REPLACE_WITH_HASH', 'Anita Sharma', 'anita@example.com',
'ADMIN'),
('rahul_m', 'REPLACE_WITH_HASH', 'Rahul Verma', 'rahul@example.com',
'MANAGER'),
('sneha_s', 'REPLACE_WITH_HASH', 'Sneha Iyer',      'sneha@example.com',
'STAFF'),
('viewer1', 'REPLACE_WITH_HASH', 'Karan Mehta',  'karan@example.com',
'VIEWER');

```

```

INSERT INTO warehouses (warehouse_code, warehouse_name, city, manager_id)
VALUES
('WH-DEL', 'Delhi Central Warehouse',      'Delhi',      2),
('WH-MUM', 'Mumbai Port Warehouse',      'Mumbai',      2),
('WH-BLR', 'Bengaluru Hub Warehouse',      'Bengaluru', 2);

```

```

INSERT INTO categories (category_name) VALUES
('Electronics'), ('Stationery'), ('Home Appliances');

```

```

INSERT INTO suppliers (supplier_name, phone) VALUES
('TechSource Traders', '90000000001'),
('PaperWorld Supplies', '90000000002');

```

```

INSERT INTO products (sku, product_name, category_id, supplier_id, unit,
reorder_level) VALUES

```

```
(
  ('LAP-001', 'Laptop 15 inch', 1, 1, 'PCS', 10), 1, 1,
  ('MOU-002', 'Wireless Mouse', 'PCS', 25), 2, 2,
  ('PAP-003', 'A4 Paper Ream', 'REAM', 50), 3, 1,
  ('MIX-004', 'Mixer Grinder', 'PCS', 8);
```

```
INSERT INTO stock (warehouse_id, product_id, quantity) VALUES
(1, 1, 60), (1, 2, 200), (1, 3, 500),
(2, 1, 15), (2, 4, 40),
(3, 2, 18), (3, 3, 120);
```

7.4 SELECT, WHERE, ORDER BY

```
-- All products with category name, sorted by name
SELECT p.product_id, p.sku, p.product_name, c.category_name
FROM products p
JOIN categories c ON c.category_id = p.category_id
ORDER BY p.product_name ASC;
```

```
-- Warehouses in a city
SELECT * FROM warehouses WHERE city = 'Delhi';
```

7.5 UPDATE and DELETE

```
-- Change the reorder level of a product
UPDATE products SET reorder_level = 12 WHERE sku = 'LAP-001';

-- Deactivate a user (soft delete is preferred over deleting history)
UPDATE users SET is_active = 0 WHERE username = 'viewer1';

-- Delete a supplier that is not used by any product
DELETE FROM suppliers
WHERE supplier_id = 2
  AND NOT EXISTS (SELECT 1 FROM products WHERE supplier_id = 2);
```

7.6 JOIN, GROUP BY, Aggregate Functions

```
-- Number of products and total quantity per category
SELECT c.category_name,
       COUNT(DISTINCT p.product_id) AS product_count,
       COALESCE(SUM(s.quantity), 0) AS total_quantity
FROM categories c
LEFT JOIN products p ON p.category_id = c.category_id
LEFT JOIN stock s    ON s.product_id = p.product_id
GROUP BY c.category_id, c.category_name
ORDER BY total_quantity DESC;

-- Highest, lowest and average stock quantity per warehouse
SELECT w.warehouse_name,
       MAX(s.quantity)      AS
       max_qty, MIN(s.quantity) AS
       min_qty, AVG(s.quantity) AS
       avg_qty
FROM warehouses w
JOIN stock s ON s.warehouse_id = w.warehouse_id
GROUP BY w.warehouse_id, w.warehouse_name;
```

7.7 Subquery

```
-- Products whose quantity in Delhi is above the Delhi average
SELECT p.product_name, s.quantity
FROM stock s
JOIN products p ON p.product_id = s.product_id
WHERE s.warehouse_id = 1
      AND s.quantity > (SELECT AVG(quantity) FROM stock WHERE warehouse_id = 1);
```

7.8 Project-Specific Queries

Query 1 – Check available stock of a product in a warehouse

```
SELECT w.warehouse_name, p.product_name, s.quantity AS available_quantity
FROM stock s
JOIN warehouses w ON w.warehouse_id = s.warehouse_id
JOIN products p   ON p.product_id   = s.product_id
WHERE s.warehouse_id = 1 AND s.product_id = 1;
```

Query 2 – Find all products in a warehouse

```

SELECT p.sku, p.product_name, c.category_name, s.quantity
FROM stock s
JOIN products p ON p.product_id = s.product_id
JOIN categories c ON c.category_id = p.category_id
WHERE s.warehouse_id = 1
ORDER BY p.product_name;

```

Query 3 – Create a stock transfer (request with items)

```

START TRANSACTION;

INSERT INTO stock_transfers
    (transfer_code, source_warehouse_id, destination_warehouse_id,
    requested_by, remarks)
VALUES ('TR-2025-0001', 1, 3, 3, 'Restock Bengaluru mouse inventory');

SET @new_transfer_id = LAST_INSERT_ID();

INSERT INTO transfer_items (transfer_id, product_id, quantity) VALUES
(@new_transfer_id, 2, 80),
(@new_transfer_id, 3, 100);

COMMIT;

```

(Before this insert, the application checks Query 1 for every item so that the request does not exceed available stock.)

Query 4 – View pending transfers

```

SELECT t.transfer_code,
       sw.warehouse_name AS from_warehouse,
       dw.warehouse_name AS to_warehouse,
       u.full_name        AS requested_by,
       t.request_date
FROM stock_transfers t
JOIN warehouses sw ON sw.warehouse_id = t.source_warehouse_id
JOIN warehouses dw ON dw.warehouse_id = t.destination_warehouse_id
JOIN users u      ON u.user_id        = t.requested_by
WHERE t.status = 'PENDING'
ORDER BY t.request_date;

```

Query 5 – View completed transfers

```

SELECT t.transfer_code, sw.warehouse_name AS from_warehouse,
       dw.warehouse_name AS to_warehouse, t.received_date
FROM stock_transfers t
JOIN warehouses sw ON sw.warehouse_id = t.source_warehouse_id
JOIN warehouses dw ON dw.warehouse_id = t.destination_warehouse_id
WHERE t.status = 'RECEIVED'
ORDER BY t.received_date DESC;

```

Query 6 – View transfer history (all statuses, with items)

```

SELECT t.transfer_code, t.status, p.product_name, ti.quantity,
       sw.warehouse_name AS from_warehouse, dw.warehouse_name AS
to_warehouse,
       t.request_date
FROM stock_transfers t
JOIN transfer_items ti ON ti.transfer_id = t.transfer_id
JOIN products p ON p.product_id = ti.product_id
JOIN warehouses ON sw.warehouse_id = t.source_warehouse_id
sw JOIN ON dw.warehouse_id = t.destination_warehouse_id
dw
ORDER BY t.request_date DESC, t.transfer_code;

```

Query 7 – Calculate total stock by warehouse

```

SELECT w.warehouse_name,
       COUNT(s.product_id) AS distinct_products,
       COALESCE(SUM(s.quantity), 0) AS total_units
FROM warehouses w
LEFT JOIN stock s ON s.warehouse_id = w.warehouse_id
GROUP BY w.warehouse_id, w.warehouse_name
ORDER BY total_units DESC;

```

Query 8 – Find low-stock products

```

SELECT w.warehouse_name, p.product_name, s.quantity, p.reorder_level
FROM stock s
JOIN products p ON p.product_id = s.product_id
JOIN warehouses w ON w.warehouse_id = s.warehouse_id
WHERE s.quantity <= p.reorder_level
ORDER BY s.quantity ASC;

```

Query 9 – Find products transferred between warehouses

```

SELECT sw.warehouse_name AS from_warehouse,
       dw.warehouse_name AS to_warehouse,
       p.product_name,
       SUM(ti.quantity) AS total_transferred
FROM stock_transfers t
JOIN transfer_items ti ON ti.transfer_id = t.transfer_id
JOIN products p ON p.product_id = ti.product_id
JOIN warehouses ON sw.warehouse_id = t.source_warehouse_id
sw JOIN ON dw.warehouse_id = t.destination_warehouse_id
WHERE sw.status = 'RECEIVED'
GROUP BY sw.warehouse_name, dw.warehouse_name, p.product_name
ORDER BY total_transferred DESC;

```

Query 10 - Update stock after a successful transfer

Approval (by the manager):

```

UPDATE stock_transfers
SET status = 'APPROVED', approved_by = 2, approval_date = NOW()
WHERE transfer_id = 1 AND status = 'PENDING';

```

Dispatch (stock leaves the source warehouse). The transaction is shown for one item; the application repeats the `UPDATE` and `INSERT` for each row of `transfer_items`.

```

START TRANSACTION;

SELECT status FROM stock_transfers WHERE transfer_id = 1 FOR UPDATE;

UPDATE stock
SET quantity = quantity - 80
WHERE warehouse_id = 1 AND product_id = 2 AND quantity >= 80;

INSERT INTO stock_transactions
    (product_id, warehouse_id, transfer_id, txn_type, quantity_change,
    created_by)
VALUES (2, 1, 1, 'TRANSFER_OUT', -80, 3);

UPDATE stock_transfers
SET status = 'IN_TRANSIT', dispatched_by = 3, dispatch_date = NOW()
WHERE transfer_id = 1 AND status = 'APPROVED';

COMMIT;

```

Receive (stock enters the destination warehouse). `ON DUPLICATE KEY UPDATE` adds to the existing row, or the insert creates a row if the product was not yet stocked there.

```

START TRANSACTION;

INSERT INTO stock (warehouse_id, product_id, quantity)
VALUES (3, 2, 80)
ON DUPLICATE KEY UPDATE quantity = quantity + VALUES(quantity);

INSERT INTO stock_transactions
    (product_id, warehouse_id, transfer_id, txn_type, quantity_change,
    created_by)
VALUES (2, 3, 1, 'TRANSFER_IN', 80, 3);

UPDATE stock_transfers
SET status = 'RECEIVED', received_by = 3, received_date = NOW()
WHERE transfer_id = 1 AND status = 'IN_TRANSIT';

COMMIT;

```

If any statement fails, the application issues `ROLLBACK`, so that nothing is changed.

7.9 Other Useful Statements

```

-- Quantity currently in transit to each warehouse
SELECT dw.warehouse_name, p.product_name, SUM(ti.quantity) AS in_transit_qty
FROM stock_transfers t
JOIN transfer_items ti ON ti.transfer_id = t.transfer_id
JOIN products p      ON p.product_id = ti.product_id
JOIN warehouses dw   ON dw.warehouse_id = t.destination_warehouse_id
WHERE t.status = 'IN_TRANSIT'
GROUP BY dw.warehouse_name, p.product_name;

-- Reject a transfer with a reason
UPDATE stock_transfers
SET status = 'REJECTED', approved_by = 2, approval_date = NOW(),
    remarks = 'Stock reserved for local orders'
WHERE transfer_id = 2 AND status = 'PENDING';

-- Demonstrate a constraint: this fails because quantity cannot be negative
UPDATE stock SET quantity = -5 WHERE stock_id = 1;

```

CHAPTER 8 – USER INTERFACE

Screenshots are not included in this report. Replace each placeholder with a real screenshot of your running application.

8.1 Login Page

- **Purpose:** to allow only registered users to enter the system.
- **Main components:** username field, password field, Login button, error message area.
- **User actions:** enter credentials and click Login.
- **Database interaction:** reads `users` to verify the account and password hash; writes a LOGIN row to `audit_logs`.
- **Expected result:** the user is taken to the dashboard; a wrong password shows "Invalid username or password".

[INSERT SCREENSHOT HERE – Login Page]

8.2 Dashboard

- **Purpose:** to give a quick summary of the whole system.
- **Main components:** summary cards (total products, total stock units, pending transfers, in-transit transfers, low-stock items), table of recent transfers, navigation menu.
- **User actions:** view summaries, click a card to open the related page.
- **Database interaction:** aggregate queries on `stock`, `stock_transfers` and `products`.
- **Expected result:** current figures are shown without manual calculation.

[INSERT SCREENSHOT HERE – Dashboard]

8.3 Warehouse Management

- **Purpose:** to maintain warehouse details.
- **Main components:** warehouse table, Add Warehouse form (code, name, city, address, manager), Edit and Deactivate buttons.
- **User actions:** add, edit, search and deactivate warehouses (Admin only).
- **Database interaction:** INSERT / UPDATE / SELECT on `warehouses`; reads `users` for manager list.
- **Expected result:** the warehouse list is updated; duplicate codes are rejected.

[INSERT SCREENSHOT HERE – Warehouse Management]

8.4 Product Management

- **Purpose:** to maintain the product catalogue.
- **Main components:** product table, Add Product form (SKU, name, category, supplier, unit, reorder level), category dropdown.
- **User actions:** add, edit and search products and categories.

- **Database interaction:** INSERT / UPDATE / SELECT on products, categories, suppliers .
- **Expected result:** new products are available for stock entry and transfers; duplicate SKUs are rejected.

[INSERT SCREENSHOT HERE – Product Management]

8.5 Inventory (Stock Management)

- **Purpose:** to show and update stock per warehouse.
- **Main components:** warehouse selector, stock table (SKU, product, quantity, reorder level), Add Stock form, search box, low-stock highlight.
- **User actions:** select a warehouse, search or filter products, add stock.
- **Database interaction:** SELECT on stock joined with products; INSERT or UPDATE on stock; INSERT on stock_transactions.
- **Expected result:** the stock table reflects the new quantity and a STOCK_IN transaction is recorded.

[INSERT SCREENSHOT HERE – Inventory]

8.6 Create Stock Transfer

- **Purpose:** to request movement of products from one warehouse to another.
- **Main components:** source warehouse dropdown, destination warehouse dropdown, product rows (product, available quantity, quantity to transfer), remarks box, Submit button.
- **User actions:** choose warehouses, add products, enter quantities, submit.
- **Database interaction:** reads stock to check availability; inserts into stock_transfers and transfer_items; writes to audit_logs.
- **Expected result:** a transfer with status PENDING and a code such as TR-2025-0001 is created. If the quantity is more than available, an "Insufficient stock" message is shown and nothing is saved.

[INSERT SCREENSHOT HERE – Create Stock Transfer]

8.7 Transfer Tracking

- **Purpose:** to follow each transfer and perform the next action.
- **Main components:** transfer table with status badges, filters (status, warehouse, date), details panel, action buttons (Approve, Reject, Dispatch, Receive, Cancel).
- **User actions:** filter transfers; the manager approves or rejects; staff dispatch and receive.

- **Database interaction:** UPDATE on stock_transfers; UPDATE on stock; INSERT on stock_transactions and audit_logs (in transactions).
- **Expected result:** status changes in order; stock changes at dispatch (source) and at receipt (destination).

[INSERT SCREENSHOT HERE – Transfer Tracking]

8.8 Reports

- **Purpose:** to support decisions with summaries.
- **Main components:** report selector (stock by warehouse, low stock, transfers between warehouses, transaction history), date filters, table view.
- **User actions:** choose a report and filters, view results.
- **Database interaction:** GROUP BY, JOIN and aggregate queries from Chapter 7.
- **Expected result:** accurate, up-to-date report tables.

[INSERT SCREENSHOT HERE – Reports]

CHAPTER 9 – TESTING

9.1 Testing Strategy

Testing was planned at four levels, from small units to the complete system, followed by acceptance testing by users. Test data uses the sample warehouses (Delhi, Mumbai, Bengaluru) and products from Chapter 7.

9.2 Unit Testing

Each function is tested separately: password check, quantity validation, stock availability check, transfer-code generation and status-change rules. Example: the availability function must return false when the requested quantity is greater than stock.quantity.

9.3 Integration Testing

The interaction between frontend, Flask routes and MySQL is tested. Example: submitting the Create Transfer form must create one row in stock_transfers and the correct rows in transfer_items.

9.4 System Testing

The whole workflow is tested from login to receipt of a transfer, including role restrictions and reports.

9.5 Database Testing

Tests check that constraints work: duplicate SKU rejected, negative stock rejected by CHECK, a transfer from a warehouse to itself rejected, foreign key violations rejected, and transactions rolled back when one statement fails.

9.6 User Acceptance Testing

Sample users (classmates acting as Admin, Manager and Staff) perform typical tasks and give feedback on clarity and ease of use. Record their comments here: [INSERT UAT FEEDBACK].

Table 9.1 – Test Cases

The *Actual Result* and *Status* columns are left for you to fill in after running each test on your own system.

ID	Test Scenario	Input	Expected Result	Actual Result	Status
TC01	Valid login	admin / correct password	Dashboard opens	[fill]	[Pass/Fail]
TC02	Invalid login	admin / wrong password	Error "Invalid username or password"; no session	[fill]	[Pass/Fail]
TC03	Add warehouse	Code WH-HYD, name Hyderabad Depot, city Hyderabad	Warehouse saved and listed	[fill]	[Pass/Fail]
TC04	Duplicate warehouse code	Code WH-DEL again	Error: code already exists; no new row	[fill]	[Pass/Fail]
TC05	Add product	SKU TAB-005, name Tablet 10 inch, category Electronics	Product saved	[fill]	[Pass/Fail]
TC06	Duplicate product SKU	SKU LAP-001 again	Error: SKU already exists	[fill]	[Pass/Fail]
TC07	Add stock	Delhi, LAP-001, quantity 20	Delhi laptop stock increases by 20; STOCK_IN row logged	[fill]	[Pass/Fail]
TC08	Create transfer	Delhi → Bengaluru, Wireless Mouse, 80	Transfer created with status PENDING	[fill]	[Pass/Fail]
TC09	Insufficient stock	Mumbai → Delhi, Laptop, 500 (Mumbai has 15)	Error "Insufficient stock"; no transfer saved	[fill]	[Pass/Fail]
TC10	Same source and destination	Delhi → Delhi		[fill]	[Pass/Fail]
TC11	Invalid product	Product ID 9999 in transfer	Error: product not found	[fill]	[Pass/Fail]

ID	Test Scenario	Input	Expected Result	Actual Result	Status
TC12	Invalid warehouse	Source warehouse ID 9999	Error: warehouse not found	[fill]	[Pass/Fail]
TC13	Approve transfer	Manager approves PENDING transfer	Status APPROVED;	[fill]	[Pass/Fail]
TC14	Reject transfer	Manager rejects with remark	approver and date saved Status REJECTED; stock	[fill]	[Pass/Fail]
TC15	Dispatch before approval	Dispatch a PENDING transfer	unchanged Error: transfer not	[fill]	[Pass/Fail]
TC16	Dispatch transfer	Dispatch APPROVED transfer	approved	[fill]	[Pass/Fail]
TC17	Automatic stock deduction	After TC16 check source stock	Status IN_TRANSIT Delhi mouse stock reduced by 80 (200 → 120)	[fill]	[Pass/Fail]
TC18	Receive transfer	Receive IN_TRANSIT transfer at destination	Status RECEIVED; received date saved	[fill]	[Pass/Fail]
TC19	Automatic destination increase	After TC18 check destination stock	Bengaluru mouse stock increased by 80 (18 → 98)	[fill]	[Pass/Fail]
TC20	Receive twice	Receive an already RECEIVED transfer	Error; stock not added again	[fill]	[Pass/Fail]
TC21	Search	Search "Mouse" in Inventory	Only matching products shown	[fill]	[Pass/Fail]
TC22	Filter transfers	Filter by status PENDING	Only pending transfers shown	[fill]	[Pass/Fail]
TC23	Reports	Open low-stock report	Products with quantity ≤	[fill]	[Pass/Fail]
TC24	Role restriction	Viewer tries to approve a transfer	reorder level listed Access denied (HTTP 403)	[fill]	[Pass/Fail]
TC25	Logout	Click Logout, then press browser Back	Session ended; login page shown	[fill]	[Pass/Fail]

CHAPTER 10 – RESULTS AND DISCUSSION

The results below are the **expected outcomes** of using the system. Replace or support them with real screenshots and observations from your own testing.

Area	Expected result
Improved stock visibility	Managers see stock of every warehouse on one screen instead of calling each location
Faster stock transfers	Requests, approvals and receipts happen online, so waiting time for paper slips is removed
Reduced manual errors	Validation and constraints stop negative stock, wrong products and invalid warehouses
Centralized database	One MySQL database is the single source of truth
Better warehouse coordination	Both source and destination see the same transfer status
Accurate stock quantities	Stock changes only inside transactions at dispatch and receipt
Transfer tracking	Every transfer shows its status, people involved and dates
Better decision-making	Low-stock and transfer reports show where stock is needed

Discussion. The workflow keeps stock accurate by separating *request*, *approval*, *dispatch* and *receipt*. Deducting stock at dispatch and adding it at receipt means goods in transit are never counted twice. The audit and transaction tables allow any quantity to be traced back to the action that caused it.

[INSERT SCREENSHOT HERE – Stock before transfer]

[INSERT SCREENSHOT HERE – Transfer in transit]

[INSERT SCREENSHOT HERE – Stock after transfer received]

[INSERT SCREENSHOT HERE – Transaction history]

CHAPTER 11 – SECURITY

Area	How it is handled
Authentication	Users must log in with a username and password; the session stores the user ID and role. Disabled users (<code>is_active = 0</code>) cannot log in.

Area	How it is handled
Authorization	Every route checks the role stored in the session before running.
Password protection	Passwords are never stored as plain text; only a salted hash is stored in <code>password_hash</code> .
SQL injection prevention	All queries use parameterised statements (%s placeholders) so user input is never joined into the SQL text.
Input validation	Required fields, data types, positive quantities and valid IDs are checked on the server, not only in the browser.
Role-based access	Admin: everything; Manager: approve, reject, manage products and stock; Staff: stock entry, request, dispatch, receive; Viewer: read-only.
Database constraints	PRIMARY KEY, UNIQUE, NOT NULL, CHECK and FOREIGN KEY constraints block invalid data even if application code has a bug.
Transaction management	Dispatch and receive use START TRANSACTION, COMMIT and ROLLBACK, so stock is either fully updated or not changed at all
Audit logs	(atomicity). Logins and important audit logs with user, action and time. Stock actions are saved in <code>stock_transactions</code> .
Backup and recovery	Regular backups with mysqldump (or MySQL Workbench export) are recommended, and a backup can be restored with the mysql command. Backup scheduling is an operational practice, not a built-in feature of the application.

Example backup and restore commands:

```
mysqldump -u root -p warehouse_transfer_db > warehouse_transfer_backup.sql
mysql -u root -p warehouse_transfer_db < warehouse_transfer_backup.sql
```

CHAPTER 12 – ADVANTAGES AND LIMITATIONS

12.1 Advantages

- **Centralized inventory:** one database for all warehouses.
- **Real-time stock visibility:** current stock is visible as soon as a transaction is committed.
- **Multiple warehouse support:** new warehouses can be added without changing the design.
- **Automated transfer tracking:** each transfer has a status and history.
- **Reduced errors:** validation, constraints and automatic updates replace manual calculations.
- **Better reporting:** reports are produced directly from SQL queries.
- **Scalable database:** a normalized design on MySQL can handle growth in products, warehouses and transfers.

12.2 Limitations

- A network connection to the server is needed because the system is web-based.
- The system depends on correct data entry by users, for example the correct quantity at stock entry.
- The database needs regular maintenance, backups and monitoring.
- The basic version does not include barcode or RFID scanning.
- The basic version does not include advanced forecasting or automatic reordering.
- Transport cost, vehicle tracking and partial receipts or damaged goods handling are not covered.

CHAPTER 13 – FUTURE ENHANCEMENTS

The following features are **not part of the current implementation**. They are possible improvements.

Enhancement	Description
Barcode scanning	Scan products at dispatch and receipt to reduce typing errors
QR code support	QR labels on transfers for quick lookup
RFID integration	Automatic reading of tagged goods

Enhancement	Description
Mobile application	Android/iOS app for warehouse staff
AI-based demand forecasting	Predict demand per warehouse from past transactions
Automatic reorder alerts	Notify when stock falls below reorder level
Cloud deployment	Host the application and database on a cloud platform
Email / SMS notifications	Inform managers of pending approvals and arrivals
Advanced analytics	Charts, trends and transfer-time analysis
Role-based dashboards	Different dashboard for each role
Supplier integration	Link purchase orders and supplier deliveries
ERP integration	Exchange data with accounting and sales systems
Real-time notifications	Live updates in the browser when a transfer changes status
Partial receipt and damage handling	Record received quantity different from dispatched quantity

CHAPTER 14 – CONCLUSION

The **Multi-Warehouse Stock Transfer Management System** addresses a common problem of businesses with several warehouses: stock moves between locations, but the records do not move with it. By storing all warehouses, products and stock in one MySQL database, the system gives every user the same, current view of inventory.

The controlled transfer life cycle – request, validation, approval, dispatch, in-transit, receipt – makes sure that only valid transfers are made and that responsibility is clear at every step. Stock is deducted from the source warehouse at dispatch and added to the destination warehouse at receipt inside database transactions, so quantities remain consistent even when an error occurs.

The normalized design (up to 3NF), with keys and constraints, improves **accuracy** by preventing duplicate and invalid data. Automatic updates, search, filters and reports improve **efficiency**. The transaction history and audit log improve **transparency**, because every movement can be traced to a user and a time.

The project gave practical experience of ER modelling, relational design, normalization, SQL, transactions and web integration. With the enhancements listed in Chapter 13, it can grow into a more complete warehouse management solution.

REFERENCES

1. *MySQL 8.0 Reference Manual*. Oracle Corporation. (Official MySQL documentation, available on the MySQL website.)
2. *Flask Documentation*. Pallets Projects. (Official Flask documentation.)
3. *Python Documentation*. Python Software Foundation. (Official Python documentation.)
4. *MDN Web Docs* – HTML, CSS and JavaScript references. Mozilla.
5. Silberschatz, A., Korth, H. F., Sudarshan, S. *Database System Concepts*. McGraw-Hill Education.
6. Elmasri, R., Navathe, S. B. *Fundamentals of Database Systems*. Pearson.
7. Ramakrishnan, R., Gehrke, J. *Database Management Systems*. McGraw-Hill Education.
8. Pressman, R. S. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
9. Sommerville, I. *Software Engineering*. Pearson.

(Add the edition and year of the copies you actually used.)

APPENDIX

Appendix A – Important SQL Commands

The complete `CREATEDATABASE`, `CREATE TABLE`, `INSERT` and query commands are given in Chapter 7. Quick reference:

Purpose	Command pattern
Create database	<code>CREATE DATABASE warehouse_transfer_db;</code>
Use database	<code>USE warehouse_transfer_db;</code>
Show tables	<code>SHOW TABLES;</code>
Describe table	<code>DESCRIBE stock;</code>
Start / finish transaction	<code>START TRANSACTION; ... COMMIT;</code>
Undo transaction	<code>ROLLBACK;</code>

Purpose	Command pattern
Backup	<code>mysqldump -u root -p warehouse_transfer_db > backup.sql</code>

Appendix B – Database Schema Summary

Table	Primary key	Foreign keys
users	user_id	–
warehouse	warehouse_id	manager_id → users
categories	category_id	–
suppliers	supplier_id	–
products	product_id	category_id → categories; supplier_id → suppliers warehouse_id → warehouses;
stock	stock_id (unique: warehouse_id + product_id)	product_id → products source_warehouse_id, destination_warehouse_id → warehouses; requested_by, approved_by, dispatched_by, received_by → users
stock_transfers	transfer_id	transfer_id → stock_transfers; product_id → products
transfer_items	transfer_item_id (unique: transfer_id + product_id)	product_id → products; warehouse_id → warehouses; transfer_id → stock_transfers;
stock_transactions	transaction_id	created_by → users user_id → users
audit_logs	log_id	

Appendix C – Sample Records

warehouses

warehouse_id	warehouse_code	warehouse_name	city
1	WH-DEL	Delhi Central Warehouse	Delhi

warehouse_i	warehouse_code	warehouse_name	city
d 2	WH-MUM	Mumbai Port Warehouse	Mumbai
3	WH-BLR	Bengaluru Hub Warehouse	Bengaluru

products

product_i	sku LAP-	product_name	category	reorder_level
d 1	001	Laptop 15 inch	Electronics	10
2	MOU-	Wireless Mouse	Electronics	25
3	002 PAP-	A4 Paper Ream	Stationery	50
4	003 MIX-	Mixer Grinder	Home	8

004 Appliances

stock (before the sample transfer)

warehous	product	quantit
e Delhi	Laptop 15 inch	y 60
Delhi	Wireless Mouse	200
Delhi	A4 Paper Ream	500
Mumbai	Laptop 15 inch	15
Mumbai	Mixer Grinder	40
Bengaluru	Wireless Mouse	18
Bengaluru	A4 Paper Ream	120

Sample transfer TR-2025-0001 (Delhi → Bengaluru): Wireless Mouse 80

Step	Status	Delhi mouse	Bengaluru mouse
Requested	PENDING	stock 200	stock 18
Approved	APPROVED	200	18
Dispatched	IN_TRANSIT	120	18
Received	RECEIVED	120	98

Appendix D – Important Source Code Snippets

The login and dispatch snippets are in Chapter 6 (Section 6.2). Add other snippets from your project here, for example the create-transfer route, the receive route and the frontend `fetch()` call:

```
async function loadPendingTransfers() {  
  const res = await fetch('/api/transfers?status=PENDING');  
  if (!res.ok) { alert('Could not load transfers'); return; }  
  const transfers = await res.json();  
  // render transfers into the table  
}
```

Appendix E – API Endpoints

See the table in Chapter 6, Section 6.2.

Appendix F – Screenshots

[INSERT SCREENSHOT HERE]

[INSERT SCREENSHOT HERE]

[INSERT SCREENSHOT HERE]